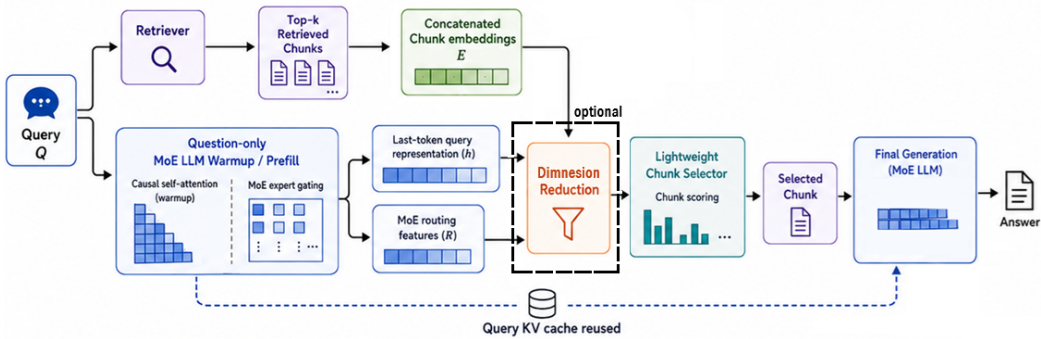


Graphical Abstract

Lightweight Chunk Selection for Mobile Retrieval-Augmented Generation

Sicong Chang, Yidan Shen, Wen Yu, Jiefu Chen, Xin Fu, Renjie Hu



Highlights

Lightweight Chunk Selection for Mobile Retrieval-Augmented Generation

Sicong Chang, Yidan Shen, Wen Yu, Jiefu Chen, Xin Fu, Renjie Hu

- Formulated mobile RAG chunk selection as a lightweight evidence-alignment problem.
- Proposed a chunk selection model using hidden states, MoE routing signals, and chunk embeddings.
- Introduced semantic chunk-correctness labels based on evidential sufficiency.
- Provided a budget-aware feature selection strategy for resource-constrained deployment settings.
- Achieved the best rank-1 chunk-selection accuracy across TriviaQA, PopQA, and MS MARCO.

Lightweight Chunk Selection for Mobile Retrieval-Augmented Generation

Sicong Chang^{a,1}, Yidan Shen^{a,1}, Wen Yu^b, Jiefu Chen^a, Xin Fu^a, Renjie Hu^{b,*}

^aDepartment of Electrical and Computer Engineering, University of Houston, Houston, TX, USA

^bDepartment of Information Science Technology, University of Houston, Houston, TX, USA

Abstract

Retrieval-augmented generation (RAG) improves the factual grounding of large language models (LLMs) by incorporating external knowledge, but deploying RAG on mobile and edge devices remains challenging because retrieved context increases computation and memory. A direct way to reduce this cost is to retain only one retrieved chunk before generation, but the top-ranked retrieved chunk is not always the most evidence-supporting one, since retrieval similarity does not necessarily imply evidential sufficiency. Existing context-reduction methods, including prompt or token-level compression, and cross-encoder reranking, can improve context quality, but often require additional LLMs or compressors that are costly under a strict mobile budget. In this paper, we study lightweight RAG chunk selection as an evidence-alignment problem. Our selector combines three complementary feature sources: question hidden states that represent LLM-side query intent, MoE routing-derived expert signals that capture the generator’s internal routing structure, and retrieved chunk embeddings that preserve candidate-side evidence geometry. A compact multilayer perceptron maps these features to an evidence prototype in the chunk embedding space, and the candidate most aligned with this prototype is selected by cosine similarity. For stricter deployment budgets, we further introduce an optional task-aware

*Corresponding author.

Email address: `rhu7@central.uh.edu` (Renjie Hu)

¹Equal contribution.

feature selection strategy to reduce the selector input dimension. To support supervised evaluation, we construct semantic chunk-correctness labels based on evidence sufficiency rather than answer-string containment. Experiments on TriviaQA, PopQA, and MS MARCO Passage Ranking show that the proposed selector consistently improves rank-1 evidence selection over mobile-applicable baselines by an average of 2.5 percentage points. These results suggest that using LLM-side query representations and MoE routing information and aligning them with retrieval-side candidate embeddings is an effective and parameter-efficient strategy for mobile-applicable RAG chunk selection.

Keywords: Retrieval-Augmented Generation, Chunk Selection, Context Reduction, Mixture-of-Experts, Edge AI

1. Introduction

Retrieval-augmented generation (RAG) has become a common framework for improving factual grounding by connecting large language models (LLMs) with external knowledge sources [1]. It is especially useful in applications that require private data access, domain-specific knowledge, or timely information updates. Meanwhile, sparse mixture-of-experts (MoE) architectures have become increasingly prominent in recent LLM families [2, 3, 4]. There is also growing interest in deploying RAG systems on mobile and edge devices, where inference is constrained by computation, memory, latency, and energy budgets. In these settings, retrieved context increases prefill computation, memory use, and end-to-end latency, making context management an important issue for resource-constrained RAG [5].

A direct way to reduce the context cost is to restrict the generator to a single retrieved chunk. This aggressive context-reduction strategy can substantially reduce prompt length and token-level computation, but it turns chunk selection into a critical bottleneck. The top-ranked retrieved chunk is not necessarily the best evidence chunk, because dense retrieval optimizes semantic similarity rather than evidential sufficiency [6]. Retrieved passages may be topically related to the query, mention the same entity, or share surface expressions without containing the fact or relation needed to answer the question. Thus, efficient mobile RAG requires a lightweight selector that identifies the most evidence-supporting chunk from the retrieved candidate pool instead of directly relying on retrieval rank.

Existing methods for reducing retrieved context can be broadly grouped into prompt compression, token- or sentence-level pruning, and chunk-level reranking. Prompt compression and rewrite-based methods reduce the final input length, but they usually require an additional LLM or compressor before generation such as LLMLingua [7, 8] and RECOMP [9]. Token-level pruning methods such as Provence are more targeted to RAG context reduction, but still introduce an additional large reranker-pruner model [10]. Cross-encoder rerankers can provide strong estimation, but they jointly encode each query-chunk pair therefore add inference cost for every candidate chunk [11]. Recent rerankers such as BGE-reranker-v2-m3 contain hundreds of millions of parameters [12], which makes them strong server-side baselines but less aligned with a strict mobile additional-module budget. Under a constrained edge-device budget, the most practical alternatives are retrieval-order methods, lexical rerankers, classical learning-to-rank models, and compact neural interaction models. These methods avoid an additional large LLM or compressor, but their relevance scores are derived primarily from surface matching or retriever-side query–document interactions. They therefore estimate passage relevance independently of the generator that will later consume the selected evidence. This generator-independent scoring creates a mismatch: a passage may be semantically related to the query while still lacking the specific fact or relation required by the downstream generator.

We address this mismatch by reusing a generator-native representation of the question. Unlike the query embedding produced by a separate retrieval encoder, the final question hidden state is generated by the same LLM that will consume the selected evidence. Recent work has shown that LLM hidden states can be projected directly into retrieval embedding spaces, indicating that they retain retrieval-relevant information [13]. We therefore use the final-token hidden state to represent the generator’s semantic interpretation of the query.

For MoE language models, routing behavior provides an additional structured representation of the query. Prior work has shown that MoE routing weights can serve as embedding-like representations and provide information complementary to hidden-state embeddings [14, 15, 16, 17, 18]. More broadly, analysis indicates that expert use patterns encode structured information about processed input [19, 20, 21]. Motivated by these findings, we use the final question hidden state and routing-weighted expert-response features as complementary generator-side signals. The hidden state summarizes the resulting query representation, while the routing-derived feature retains

structured information about expert utilization across layers.

These generator-side features characterize the information need encoded by the language model, but they do not represent the differences among the retrieved candidates. We therefore combine them with dense chunk embeddings, which provide compact candidate-specific representations in a shared retrieval space and use a multilayer perceptron (MLP) to predict an evidence prototype in the same embedding space. Each candidate is then scored by the cosine similarity between its embedding and the predicted prototype. For more resource-constrained settings, we further introduce an optional task-aware feature-selection module that retains the most informative input dimensions before evidence-prototype prediction. In the RAG pipeline, retrieval proceeds in parallel with the question-only language model warmup, and the resulting query-side KV cache is reused during subsequent generation. The remaining online selection cost is therefore limited to the compact selector and cosine-similarity scoring.

Another challenge is the lack of reliable chunk-level supervision. Existing open-domain QA datasets provide final answers, but they do not directly specify which retrieved chunk should be retained before generation. A common lexical heuristic is to mark a chunk as correct if it contains the reference answer string, but this criterion is noisy for RAG chunk selection: a chunk may mention the answer string without providing the relation needed to answer the question, while another chunk may support the answer through paraphrase, aliasing, or indirect description. We therefore define chunk correctness by evidence sufficiency rather than answer-string containment, enabling supervised training and evaluation of pre-generation chunk selection.

Across TriviaQA, PopQA, and MS MARCO datasets, the proposed selector achieves the highest rank-1 chunk-selection accuracy among the evaluated lightweight baselines, with an average improvement of 2.49 percentage points over the strongest baseline. The feature-budget analysis further shows that reduced-input variants retain competitive performance under tighter selector budgets.

The contributions of this paper are as follows:

- We formulate resource-constrained RAG chunk selection as a LLM-coupled evidence selection problem, where query-side features are reused from the language-model warmup stage.
- We propose a lightweight chunk selection model that combines question hidden states, routing derived expert signals, and retrieved chunk

embeddings in a compact multilayer perceptron.

- We show that our proposed method outperforms current mobile-RAG methods across three different datasets using only a compact trainable selection module.
- We introduce a chunk correctness dataset for RAG chunk selection, which is designed to study efficient context reduction under a supervised setting.

2. Related Work

Retrieval-augmented generation (RAG) systems often add a post-retrieval stage to reduce input length, suppress retrieval noise, and lower generation latency. Existing approaches can be organized into four categories: (i) token-level pruning or compression, (ii) structured extraction and selection, (iii) rewrite-based condensation that summarizes or synthesizes across top retrieved contexts, (iv) lightweight lexical and neural reranking.

2.1. Token-Level Context Compression

Token-level methods shrink the generator input by deleting or compressing lexical units under a token budget. *Selective Context* identifies redundancy via language-model-based token scoring and prunes low-utility regions, yielding substantial reductions in inference memory and latency while maintaining downstream quality [22]. Prompt-compression approaches such as *LLMLingua* perform coarse-to-fine token removal with a budget controller to preserve semantics under high compression ratios [23], and *LLMLingua-2* further formulates compression as token classification trained via LLM distillation to improve faithfulness and efficiency [8]. Query-guided attention compression leverages question-to-context attention to filter tokens under explicit length constraints [24]. Although token-level compression reduces the generator’s prompt length, it often introduces non-trivial end-to-end overhead and can delete discourse-critical cues, making the resulting fragmented context harder for small on-device LLMs to use reliably.

2.2. Structured Extraction and Selection

Structure-preserving methods keep linguistic units intact while removing irrelevant content. At the sentence/span level, *RECOMP* includes an extractive compressor that selects useful sentences and an abstractive compressor

that synthesizes across documents; it also supports selective augmentation by outputting empty context when retrieval is unhelpful [9]. *FILCO* targets imperfect retrieval by learning to filter contexts using lexical and information-theoretic signals, improving robustness across knowledge-intensive tasks [25]. In deployed stacks, embedding-based optimizers similarly remove low-similarity sentences to reduce token usage [26]. At the passage level, systems often select a subset of retrieved chunks and/or apply reranking. Modern RAG toolkits expose strong rerankers, frequently cross-encoders that jointly score each (q, p_i) pair to reorder candidates beyond embedding similarity [27]. However, reranking introduces additional inference and memory overhead, which can be costly for interactive on-device RAG.

2.3. Rewrite-Based Condensation

Rewrite-based condensing replaces retrieved text with a shorter query-aware representation. *RECOMP* includes an abstractive compressor trained for end-task utility while keeping summaries concise [9]. *CompAct* actively condenses extensive retrieved documents via iterative refinement to retain multi-hop evidence under high compression rates [28]. Hierarchical summary substitution approaches such as *RAPTOR* build multi-resolution summaries and retrieve from a tree of abstractions at inference time [29]. Tooling-level implementations also expose LLM-based extractors that rewrite/extract relevant parts of each retrieved document [30]. While effective in reducing prompt length, rewrite-based methods typically require additional LLM or extra compressor before answering, significantly increasing on-device latency and energy.

2.4. Lightweight Lexical and Neural Reranking

In addition to context compression and extraction, another line of work reorders retrieved candidates using lexical, learning-to-rank, or compact neural ranking models which are most suitable to mobile-applicable chunk selection because they avoid an additional large LLM or heavy context-compression model at inference time. BM25 remains a strong lexical ranking baseline based on term-frequency and document-length normalization under the probabilistic relevance framework [31]. Classical learning-to-rank methods such as RankSVM [32] and LambdaMART [33] learn ranking functions from supervised relevance signals and are widely used as efficient reranking baselines.

Neural interaction models further improve ranking by modeling query-document matching patterns. DUET combines local exact-match signals

with distributed semantic representations for web search ranking [34]. DRMM models ad-hoc retrieval at the query-term level using matching histograms, a feed-forward matching network, and term gating [35]. KNRM introduces kernel pooling over word-level similarity matrices to capture multi-level soft matching patterns in an end-to-end ranking model [36]. These models provide compact neural alternatives to large cross-encoder rerankers, but their relevance estimates are still derived from query-document matching alone. They do not exploit the internal representations of the generator that will produce the final answer, such as hidden states or MoE routing behavior.

3. Methodology

Our RAG method is coupled with the LLM inference pipeline. Given a user query, the retriever and the language-model warmup stage are launched in parallel. The retriever returns the top-K candidate chunks, while the LLM performs a question-only forward pass to prefill the query tokens. This warmup pass is already required by the LLM inference, and the resulting query-side KV cache can be reused after chunk selection. For MoE models, the routing patterns generated during this pass provide a layer-expert representation of how computation is allocated for the query. Prior work has shown that such routing representations contain information complementary to hidden states and can function as embedding-like features [14]. Therefore, the MoE routing features and query hidden state used by the selector are extracted from computation shared with the subsequent generation stage.

After the candidate chunks and warmup-stage query features become available, the selector estimates which retrieved chunk is most likely to provide useful evidence. The selector combines three feature sources: the final-token hidden state of the query, the MoE routing weights, and the embeddings of the retrieved candidate chunks. It then predicts an evidence vector in the chunk embedding space, and the final chunk is selected by cosine similarity between this predicted vector and each candidate chunk embedding. For more resource-constrained settings, we further introduce an optional budget-aware feature selection strategy that applies a task-aware feature mask to the extracted representation before the chunk selector, thereby reducing the selector input dimension. The overall pipeline is shown in Figure 1.

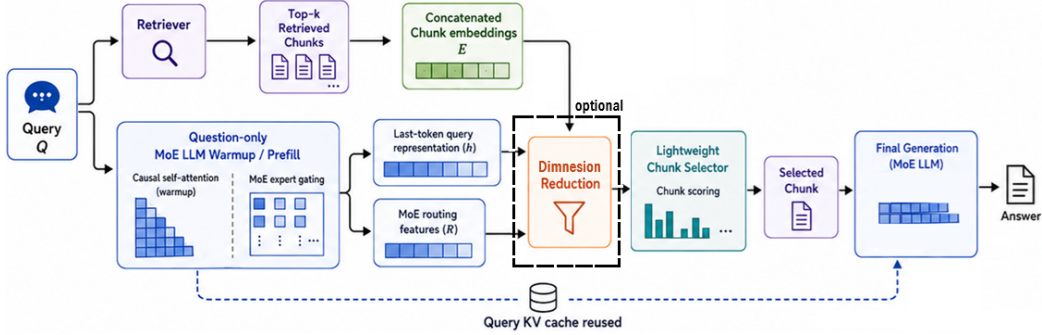


Figure 1: Model overall architecture

3.1. Input Representation

To utilize information from both the query and the retrieved candidates, we construct the input representation from three complementary feature sources: the question hidden state, the MoE routing-derived expert response, and the dense embeddings of the retrieved chunks. The question-side features describe how the language model internally represents the query, while the chunk-side features preserve the semantic structure of the candidate evidence produced by the retriever.

For each query q , the input vector is defined as

$$x = [h \parallel r \parallel E] \in \mathbb{R}^{4992}, \quad (1)$$

where $h \in \mathbb{R}^{2048}$ is the question hidden state, $r \in \mathbb{R}^{1024}$ is the routing-score feature, and $E \in \mathbb{R}^{1920}$ is the flattened representation of the top-K retrieved chunk embeddings. In this work, we use $K = 5$, and each retrieved chunk is represented by a 384-dimensional dense embedding.

The hidden state h is extracted from the final-layer last-token representation of the MoE language model under question-only input. This representation provides a compact summary of the query semantics before any retrieved context is added. Because it is obtained from a single question-only forward pass, it avoids the repeated inference required by cross-encoder reranking, where every query–chunk pair must be processed separately.

In addition to the hidden state, we use the routing behavior of the MoE model as a complementary query-side signal. The hidden state captures the final semantic representation of the query, whereas the routing pattern reflects how the model distributes the query across different experts during

inference. However, the raw router probability only represents the model’s allocation preference before expert transformation and does not directly measure the realized contribution of an expert to the resulting representation. An expert may receive a high routing probability but produce an output with a small magnitude, while another expert with a lower routing probability may exert a stronger representation-level effect through a larger response. We therefore combine the routing probability with the magnitude of the corresponding expert output.

For each layer–expert pair, we compute a routing-weighted expert-response score by averaging over all question tokens:

$$\text{RS}_{l,j} = \frac{1}{T} \sum_{t=1}^T p_{l,j}^{(t)} \left\| g_{l,j} \left(u_l^{(t)} \right) \right\|_2, \quad (2)$$

where T is the number of question tokens, $p_{l,j}^{(t)}$ is the router probability assigned to expert j at layer l for token t , $u_l^{(t)}$ denotes the input representation to the MoE block, and $g_{l,j}(\cdot)$ denotes the output of expert j . The product in Eq. 2 provides a closer approximation of the realized expert contribution than the router probability alone.

We use the routing probabilities over all 64 experts rather than retaining only the top-8 routing values. The resulting scores from all 16 layers and 64 experts per layer are flattened to form the routing feature r .

For the candidate side, each retrieved chunk c_k is represented by an ℓ_2 -normalized dense embedding $e_k \in \mathbb{R}^{384}$, where $\|e_k\|_2 = 1$. The top- K chunk embeddings are stacked and flattened as

$$E = \text{vec} \left([e_0^\top; e_1^\top; \cdots; e_{K-1}^\top] \right) \in \mathbb{R}^{384K}. \quad (3)$$

With $K = 5$, this gives $E \in \mathbb{R}^{1920}$.

Overall, h and r provide query-side information derived from a single language-model pass, while E provides the candidate-side retrieval information. The selector then learns to combine these features to estimate which retrieved chunk is most useful for downstream generation, without explicitly modeling token-level interactions between the query and every candidate chunk.

3.2. Budget-Aware Dimension Reduction

The full input representation is high-dimensional because it combines the question hidden state, routing-score feature, and retrieved-chunk embeddings. Although these features provide complementary information for chunk

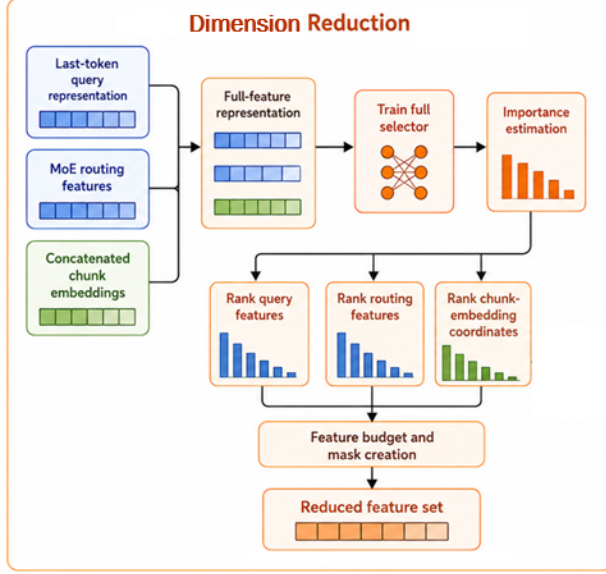


Figure 2: Budget-Aware Optional Dimension Reduction Module

selection, directly using all dimensions can be inefficient and may complicate learning. However, mobile and edge deployment may impose stricter constraints on selector size, memory bandwidth, and input dimensionality. We therefore perform an optional dimension reduction to retain the dimensions most relevant to the supervised chunk-selection objective, as shown in Figure 2.

Instead of applying an unsupervised compression method, we estimate feature importance from the trained selector. This is because variance-based compression does not necessarily preserve dimensions that are useful for evidence selection. A feature dimension with small variance may still be discriminative if the selector consistently relies on it to distinguish useful chunks from irrelevant ones. We therefore use gradient-based sensitivity as a task-aware importance measure.

Specifically, we first train a full-dimensional selector using the original input $x \in \mathbb{R}^{4992}$. After training, the selector is frozen, and the importance of each input dimension is measured by the accumulated absolute gradient of the loss with respect to that dimension:

$$I_d = \sum_{t \in \mathcal{T}} \left| \frac{\partial \mathcal{L}_t}{\partial x_d} \right|, \quad (4)$$

where $\mathcal{T}$ denotes the training set and $\mathcal{L}_t$ is the loss for training sample t . A larger I_d indicates that changes along dimension d have a stronger effect on the selection loss, and therefore that the dimension is more important for the trained selector.

For the question hidden-state and routing-score blocks, dimensions are ranked directly according to I_d . For the chunk-embedding block, we apply an additional constraint. Since all retrieved chunks are represented in the same embedding space, the same embedding coordinate should be kept or removed consistently across all chunk positions. Otherwise, different candidates would be compared using different subspaces, which could introduce an artificial position-dependent bias.

Therefore, for each chunk-embedding coordinate p , we aggregate its importance across the K retrieved chunks:

$$I_p^{\text{chunk}} = \frac{1}{K} \sum_{k=0}^{K-1} I_{E_{k,p}}, \quad (5)$$

where $I_{E_{k,p}}$ denotes the importance of coordinate p in the embedding of the k -th retrieved chunk. If coordinate p is selected, it is retained for all retrieved chunks. This preserves the shared embedding geometry used for candidate comparison.

Given a target retention ratio $X\%$, we define the feature budget as $\lfloor 4992 \cdot X/100 \rfloor$ with respect to the original input size. Selecting one hidden-state or routing-score dimension consumes one unit of this budget, while selecting one chunk-embedding coordinate consumes K units because the coordinate is retained for all K chunks. The selected dimensions form a feature mask, which is applied to the original input to produce a reduced representation $x' \in \mathbb{R}^D$.

After the mask is obtained, we retrain the MLP selector using the reduced representation x' and directly apply the same mask during inference. This two-stage process allows the full-dimensional model to first identify task-relevant dimensions, and then replaces it with a smaller selector trained only on the retained features.

3.3. Chunk Selection

Given the reduced representation x' , the selector predicts an evidence prototype in the same embedding space as the retrieved chunks:

$$\hat{y} = f_{\theta}(x') \in \mathbb{R}^{384}, \quad (6)$$

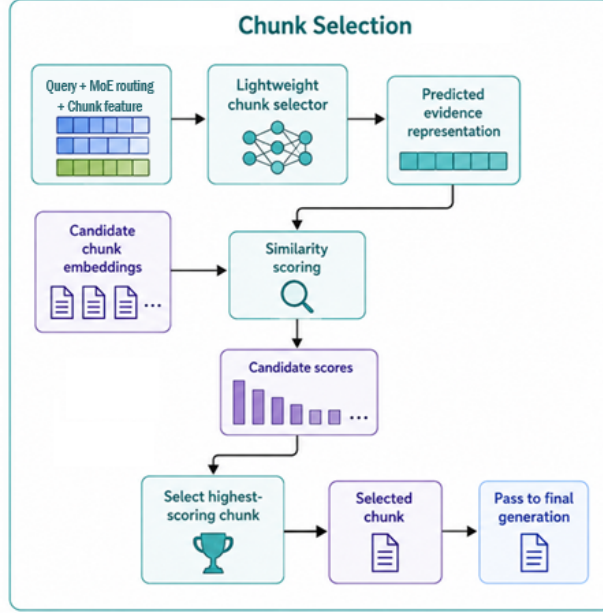


Figure 3: Chunk Selection Module

where f_θ is a compact multilayer perceptron. Rather than directly predicting a retrieval position, the selector estimates the location of supporting evidence in the retriever embedding space. This formulation allows the predicted prototype to be compared with all candidate chunks using the semantic geometry of the retriever. Although the ordered candidate set may implicitly preserve retrieval-rank information, retrieval rank is not used as the supervision target.

Before candidate scoring, the predicted prototype is ℓ_2 -normalized as $\tilde{y} = \hat{y} / \|\hat{y}\|_2$. Since each retrieved chunk embedding e_k is also normalized, its alignment with the predicted prototype is computed by cosine similarity:

$$s_k = \text{sim}(\tilde{y}, e_k) = \tilde{y}^\top e_k. \quad (7)$$

The resulting scores quantify the relative alignment of the retrieved candidates with the predicted evidence prototype and are used to prioritize the candidate set before generation.

Multi-target training objective. A query may be supported by more than one retrieved chunk. Therefore, using a single-label objective can incorrectly penalize the selector when it assigns high similarity to another chunk that

also contains valid evidence. To better match this property of RAG evidence selection, we train the selector with a multi-target contrastive loss.

For training sample i , let $\mathcal{P}_i$ denote the set of positive chunk embeddings that contain valid answer evidence, and let $\mathcal{N}_i$ denote the remaining retrieved chunk embeddings. The loss is defined as

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\sum_{p \in \mathcal{P}_i} \exp(\text{sim}(\tilde{y}_i, p)/\tau)}{\sum_{c \in \mathcal{P}_i \cup \mathcal{N}_i} \exp(\text{sim}(\tilde{y}_i, c)/\tau)}, \quad (8)$$

where τ is the temperature parameter and B is the batch size. This objective pulls the predicted vector toward any answer-supporting chunk while reducing its similarity to retrieved chunks that do not contain useful evidence.

Training samples with no positive chunks are excluded because they do not provide a supervised target for selection. Samples with no negative chunks are also excluded because they do not define a meaningful contrastive comparison among retrieved candidates.

At inference time, the same feature mask is applied to the input representation to obtain x' . The selector then predicts $\tilde{y}$, and each retrieved chunk is scored by cosine similarity. The final selected chunk is

$$k^* = \arg \max_{k \in \{0, \dots, K-1\}} \text{sim}(\tilde{y}, e_k). \quad (9)$$

The selected chunk c_{k^*} is paired with the original query for downstream generation. In this way, the selector filters retrieved context without applying token-level query–chunk interaction to every candidate.

4. Dataset and Chunk-Level Ground-Truth Construction

Our task requires supervision at the retrieved-chunk level rather than only at the final-answer level. Existing open-domain question answering datasets provide question–answer pairs, but they do not directly indicate which retrieved candidates contain sufficient evidence for answering the question. A common alternative is to derive chunk labels from answer-string containment. However, lexical containment is only an approximate indicator of evidential support and can produce both false-positive and false-negative labels.

To obtain supervision that is better aligned with chunk selection, we construct semantic support labels for the retrieved candidates. Section 4.1

describes the annotation criterion and labeling procedure, while Section 4.2 presents the resulting instance structure and its instantiation on TriviaQA, PopQA, and MS MARCO Passage Ranking.

4.1. Chunk-Level Ground-Truth Construction

Answer-string containment does not always correspond to evidential support, as illustrated in Figure 4. Two types of disagreement are particularly relevant. First, a retrieved chunk may contain the reference answer string without expressing the relation needed to answer the question. For example, the answer may occur in a list, a biographical note, a disambiguation passage, or an otherwise unrelated discussion. Such cases are false positives under lexical labeling. Second, a chunk may provide sufficient supporting evidence without containing the exact reference string, for example through paraphrases, aliases, or distributed descriptions. These cases are false negatives under lexical labeling.

To address these limitations, we construct semantic support labels for the retrieved candidate chunks. Each question–chunk pair is evaluated using GPT-5.1 as an automated annotator. The annotator is asked to determine whether the chunk provides sufficient evidence to answer the question using only the supplied context. This procedure defines chunk correctness by evidential sufficiency rather than exact answer-string occurrence.

Prompt template.

```
Can you find the evidence in the context to answer the question?  
Answer only with Yes/No. Don't use any external tool. Don't search  
on web. Only based on context, don't infer.  
Question: {question}  
Context: {context}
```

A chunk is labeled positive if it contains enough information to answer the given question. A chunk is labeled negative if it does not provide relevant evidence, even when the answer string appears in the chunk. In a preliminary comparison, the original answer-containment labels agree with the semantic support labels for 78% of the evaluated chunks, indicating that lexical matching is useful but introduces substantial label noise. The semantic verification process therefore serves two purposes: it recovers supporting chunks missed by lexical matching, and it removes spurious positives caused by accidental answer-string overlap. The resulting semantic support labels serve as the ground truth for training and evaluating the chunk selector.

Examples of disagreements between the string-match and GPT semantic label

String matching can mark unsupported chunks as positive and miss supporting chunks that lack the exact answer string.

✗ False Positives from String Matching

The answer string appears, but the chunk does not actually support the answer.

1 Q: Paper size with dimensions 297×420 mm?
A: A3
Chunk: ... paper sizes included in CSS: A3, A4, A5, B4, B5 ...

✗ Listed answer, missing supporting relation

2 Q: Force keeping the moon in orbit?
A: Gravity
Chunk: ... when a body is acted on by the gravity of another body ...

✗ Answer appears in unrelated context

3 Q: Snooker ball scoring 1 point?
A: Red
Chunk: ... the remaining red ball is placed halfway between ...

✗ Answer appears in unrelated context

-- False Negatives from String Matching

The chunk supports the answer, but does not contain the exact answer string.

1 Q: Brian Cox's 90s band?
A: D-REAM
Chunk: ... he joined D:Ream, a group that had several hits ...

-- Surface-form mismatch

2 Q: Boxcars in craps, what numbers?
A: Sixes
Chunk: ... Boxcars ... getting a 6 on each die ...

-- Surface-form mismatch

3 Q: Two spices from the nutmeg tree?
A: Nutmeg and Mace
Chunk: ... we look at nutmeg ... dried to create mace ...

-- Answer distributed across the chunk

Figure 4: Examples of disagreements between the string-match `contains_answer` label and the GPT-5.1 semantic support label on TriviaQA.

4.2. Dataset Structure and Instantiation

Each dataset instance is constructed from a question and reference answer drawn from the corresponding question answering benchmark, together with five candidate chunks retrieved from a separate passage corpus. Candidate chunks are annotated with semantic support labels indicating whether they provide sufficient evidence for the reference answer. An instance is considered answerable if at least one retrieved chunk receives a positive semantic support label. Otherwise, it is treated as unanswerable within the retrieved set.

TriviaQA. TriviaQA is an open-domain question answering benchmark constructed from Wikipedia articles. Questions typically focus on encyclopedic facts, entities, events, and concepts that can often be answered using a single supporting passage. For each TriviaQA question, candidate chunks are retrieved from a static index of DPR-style 100-word Wikipedia passages, `psgs_w100`, containing approximately 21 million passages. Since the underlying retrieval corpus is relatively clean and well structured, TriviaQA

provides a representative benchmark for evaluating evidence selection in a controlled retrieval environment.

PopQA. PopQA is an entity-focused question answering benchmark built from knowledge graph relations. The dataset covers facts associated with entities spanning a wide range of popularity levels, including both frequently mentioned and long-tail entities. Compared with traditional Wikipedia-based benchmarks, PopQA places greater emphasis on retrieving evidence for less common facts, making it useful for evaluating robustness across different levels of knowledge frequency. For each PopQA question, candidate chunks are retrieved from the same static index of DPR-style 100-word Wikipedia passages used for TriviaQA.

MS MARCO Passage Ranking. MS MARCO is a large-scale retrieval benchmark derived from real-world search queries and web documents. In contrast to the Wikipedia-based retrieval corpus used for TriviaQA and PopQA, its passages originate from diverse web sources and exhibit substantial variation in writing style, document quality, and relevance. We use the MS MARCO Passage Ranking task (MS MARCO PR), with candidate chunks retrieved from the official MS MARCO passage collection, containing approximately 8.8 million passages. Retrieved passages frequently contain partial answers, redundant information, or references to answer entities without providing sufficient evidence to answer the question. These characteristics make evidence identification substantially more challenging, resulting in a more difficult chunk selection task than in TriviaQA or PopQA.

For all datasets, semantic support labels are constructed using the procedure described in Section 4.1. Given a question and its retrieved candidate chunks, chunk selection is formulated as a supervised prediction problem in which the model identifies the candidate most likely to provide sufficient evidence for answering the question. Together, TriviaQA, PopQA, and MS MARCO PR allow evaluation under increasingly challenging retrieval conditions, ranging from relatively clean encyclopedic passages to long-tail factual knowledge and diverse web documents.

5. Experimental Setup

5.1. Retrieval Configuration

Passages and queries are encoded using the `all-MiniLM-L6-v2` Sentence-Transformer model [37, 38] with ℓ_2 -normalized embeddings. For scalable

retrieval, we construct a FAISS IVF-PQ index [39] configured with 8192 clusters, 64 subquantizers, and 8 bits per subquantizer. The quantizer is trained on a uniform sample of 500,000 embeddings per corpus. During query execution, we search 32 clusters and retain the top five passages. All experiments share this uniform retrieval and candidate-generation protocol across benchmarks.

5.2. Feature Extraction and Selector Implementation

Query-side features are extracted using the frozen `OLMoE-1B-7B-0924-Instruct` model, which features 16 layers, 64 experts per layer. Forward hooks on all MoE layers collect the required hidden-state and routing information without updating the backbone parameters. Feature extraction runs on a 16GB NVIDIA Tesla V100 GPU using 4-bit NF4 and double quantization with FP16 computation. Questions are formatted using the official instruction template truncated to 512 tokens, strictly excluding retrieved chunks.

Selector inputs undergo standard scaling. The selector is a 7-million-parameter multilayer perceptron comprising three 1024-width hidden layers, each equipped with batch normalization, ReLU activation, and a dropout rate of 0.05. The final output vector is ℓ_2 -normalized prior to candidate scoring.

Optimization utilizes AdamW with a learning rate of 8×10^{-4} , weight decay of 3×10^{-5} , and a batch size of 256. We apply Kaiming normal initialization, gradient clipping at a norm of one, and a OneCycleLR schedule with cosine annealing. Models are trained for up to 150 epochs with a 20-epoch early stopping patience based on validation loss. Instances lacking either positive or negative candidates are filtered out during optimization.

6. Results

6.1. Comparison with SOTA methods

Table 1 compares the rank-1 chunk-selection accuracy of our method with seven representative mobile-RAG baselines on three datasets. The parameter column reports the number of additional trainable parameters introduced by the reranking or selection module.

Our method achieves the best performance on all three datasets, demonstrating that the proposed lightweight selector can consistently improve the quality of the top-ranked evidence chunk. Compared with the retrieval-only

Table 1: Rank-1 chunk-selection accuracy (%) under a mobile-applicable selection budget. The parameter column reports additional trainable parameters introduced by the selection module. The best result on each dataset is shown in bold.

Method	Params	TriviaQA	PopQA	MS MARCO PR
RAG baseline	–	44.96	61.74	65.82
BM25 (lexical) [31]	–	45.01	62.80	66.14
RankSVM [32]	–	46.13	64.46	67.49
LambdaMart [33]	–	46.00	64.41	66.76
Local-DUET [34]	49M	46.28	63.33	66.97
DRMM [35]	48M	46.53	65.72	68.13
KNRM [36]	48M	47.04	66.76	69.58
Ours	7M	49.56	70.87	70.43

RAG baseline, our method improves rank-1 accuracy by 4.60 points on TriviaQA, 9.13 points on PopQA, and 4.61 points on MS MARCO PR. These gains indicate that the first retrieved chunk is often not the most reliable evidence for generation, and that an explicit chunk-selection module is necessary even when the retriever already returns relevant candidates.

Compared with lexical and classical learning-to-rank methods, our method shows a clear advantage. BM25 only brings marginal improvements over the RAG baseline, suggesting that lexical matching alone cannot reliably distinguish truly supportive chunks from topically related but insufficient ones. RankSVM and LambdaMart further improve the ranking by using hand-crafted ranking features, but their gains remain limited because they do not directly model fine-grained semantic alignment between the query and the candidate evidence. In contrast, our method learns a compact neural selection function that better captures evidence-level relevance, leading to stronger rank-1 selection accuracy across all datasets.

Among prior neural baselines, our method still outperforms KNRM by 2.52, 4.11, and 0.85 points on TriviaQA, PopQA, and MS MARCO PR, with an average gain of 2.5 points, while using only 7M additional parameters. This result is important because KNRM and DRMM rely on substantially larger neural interaction modules under our parameter accounting, whereas our model achieves better accuracy with a much smaller selection network. Therefore, the improvement is not only in absolute accuracy, but also in the accuracy–efficiency trade-off, which is crucial for mobile-device RAG.

The largest gain appears on PopQA. This is reasonable because PopQA

Table 2: Ablation study of the proposed chunk selector. HS denotes hidden-state features, RW denotes routing-weight features, and Chunk denotes retrieved chunk embeddings. The best result on each dataset is shown in bold.

Model	TriviaQA	PopQA	MS MARCO PR
Rank baseline	44.96	61.74	65.82
HS	46.24	64.85	66.45
RW	44.98	62.50	65.93
Chunk	47.31	67.92	69.28
HS + RW	44.95	63.10	66.18
HS + Chunk	49.03	68.55	69.87
RW + Chunk	48.27	69.41	70.05
HS + RW + Chunk (ours)	49.56	70.87	70.43

contains many entity-centric factual questions, where multiple retrieved chunks may share similar entities or surface-level keywords, but only one chunk provides the direct factual support needed for answering. Our method is better able to identify this evidence-bearing chunk, which explains the large improvement over both lexical and neural baselines. On TriviaQA, the absolute margin is smaller because all methods perform within a narrow range, suggesting that the main bottleneck may come from candidate retrieval quality or ambiguous evidence distribution rather than reranking capacity alone. On MS MARCO PR, our method also obtains the best result, showing that the proposed selector generalizes beyond open-domain factual question-and-answer to passage-ranking-style retrieval scenarios.

6.2. Ablation Study

Table 2 presents the ablation results of the proposed chunk selector on TriviaQA, PopQA, and MS MARCO PR. Starting from the rank baseline, we progressively incorporate hidden-state features (HS), routing-weight features (RW), and retrieved chunk embeddings (Chunk) to evaluate the contribution of each component.

The results show that retrieved chunk embeddings are the most important source of performance improvement. Using chunk embeddings alone increases rank-1 accuracy from 44.96 to 47.31 on TriviaQA, from 61.74 to 67.92 on PopQA, and from 65.82 to 69.28 on MS MARCO PR. These gains are substantially larger than those obtained by HS or RW individually, indicating that semantic information encoded in the retrieved chunks provides

the strongest signal for identifying answer-supporting evidence.

The contributions of HS and RW alone are more modest. HS consistently improves performance across all datasets, suggesting that the language model hidden state captures useful information about question semantics. In contrast, RW alone provides only limited gains, indicating that routing patterns are not sufficiently informative when used in isolation. Nevertheless, routing information appears to contain complementary signals that become useful when combined with stronger semantic features.

The pairwise combinations further reveal interactions among the three feature sources. Both HS + Chunk and RW + Chunk outperform Chunk alone on all datasets, demonstrating that hidden-state and routing information provide additional evidence beyond the retrieved chunk content itself. In particular, RW + Chunk achieves 70.05 on MS MARCO PR, while HS + Chunk reaches 49.03 on TriviaQA, showing that the usefulness of complementary signals varies across datasets.

The complete model achieves the best performance on all three benchmarks, reaching 49.56 on TriviaQA, 70.87 on PopQA, and 70.43 on MS MARCO PR. Compared with the rank baseline, the full model improves rank-1 chunk-selection accuracy by 4.60 points, 9.13 points, and 4.61 points on the three datasets, respectively. The largest improvement is observed on PopQA, where multiple retrieved chunks frequently contain similar entity mentions while only a subset provides sufficient evidence for answering the question. In such cases, combining semantic chunk representations with question-aware hidden states and routing signals enables more accurate evidence identification.

Overall, the ablation study demonstrates that retrieved chunk embeddings provide the primary source of selection accuracy, while hidden-state and routing-weight features contribute complementary information. The combination of all three components yields the most robust performance across diverse retrieval settings, validating the design of the proposed chunk-selection framework.

6.3. Feature Budget Analysis

Table 3 reports the performance of the selector under different retained-feature budgets. The full-feature model achieves the best accuracy on all three datasets, reaching 49.56 on TriviaQA, 70.87 on PopQA, and 70.43 on MS MARCO PR. This result confirms that the hidden-state, routing-derived,

Table 3: Feature-budget analysis of the proposed selector. The full-feature model achieves the best accuracy, while reduced-feature models provide smaller operating points under stricter deployment budgets.

Feature Budget	# Dims	TriviaQA	PopQA	MS MARCO PR	Avg.
10%	499	45.83	66.19	67.42	59.81
20%	998	47.01	68.03	68.67	61.24
30%	1498	47.34	68.75	68.90	61.66
40%	1997	48.12	69.31	69.36	62.26
50%	2496	48.29	69.20	69.58	62.36
60%	2995	48.61	69.94	69.79	62.78
70%	3494	48.47	70.28	70.05	62.93
80%	3994	48.98	70.42	70.13	63.18
90%	4493	49.22	70.65	70.31	63.39
100%	4992	49.56	70.87	70.43	63.62

and chunk-embedding features provide complementary selection signals when the deployment budget allows.

The reduced-feature models show a clear accuracy–efficiency trade-off. Even with only 10% of the original features, the selector improves over the retrieval-only rank baseline on all three datasets, increasing accuracy by 0.87 points on TriviaQA, 4.45 points on PopQA, and 1.60 points on MS MARCO PR. This indicates that the gradient-based feature budget retains a compact subset of informative dimensions rather than discarding features uniformly. The 20% feature-budget model uses only 998 input dimensions and achieves 61.24 average accuracy, which is better than the strongest compact neural baseline KNN at 61.13 average accuracy.

As the feature budget increases, performance improves smoothly and then gradually saturates. The 70% feature-budget model reaches 62.93 average accuracy, and the 90% model reaches 63.39, close to the full-feature result of 63.62. Therefore, feature selection provides a controllable operating curve: the full selector gives the best accuracy, while reduced selectors offer competitive alternatives when input dimensionality, parameter count, or memory bandwidth is more constrained.

7. Conclusion

This paper presented a lightweight chunk-selection framework for retrieval-augmented generation. The proposed method combines the question hidden

state, routing-weighted MoE features, and retrieved chunk embeddings to select the correct chunk for LLM inference. To provide supervision for this task, we constructed semantic support labels based on evidential sufficiency rather than lexical answer containment. Across TriviaQA, PopQA, and MS MARCO Passage Ranking, the proposed selector achieved the highest rank-1 chunk-selection accuracy among the evaluated lightweight baselines, improving over the strongest baseline on each dataset by an average of 2.49 percentage points.

The optional task-aware feature-selection procedure provides additional operating points under reduced selector-input budgets. Overall, the results show that generator-side query representations can be combined with retrieval-side candidate geometry to support compact, evidence-oriented chunk selection.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive nlp tasks, in: *Advances in Neural Information Processing Systems*, Vol. 33, 2020, pp. 9459–9474.
URL <https://arxiv.org/abs/2005.11401>
- [2] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. Bou Hanna, F. Bressand, et al., Mixtral of experts, *arXiv preprint arXiv:2401.04088* (2024). doi:10.48550/arXiv.2401.04088.
URL <https://arxiv.org/abs/2401.04088>
- [3] DeepSeek-AI, DeepSeek-V3 technical report, *arXiv preprint arXiv:2412.19437* (2024). doi:10.48550/arXiv.2412.19437.
URL <https://arxiv.org/abs/2412.19437>
- [4] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al., Qwen3 technical report, *arXiv preprint arXiv:2505.09388* (2025). doi:10.48550/arXiv.2505.09388.
URL <https://arxiv.org/abs/2505.09388>
- [5] D. H. Kang, H. Lee, H. Cha, On device RAG for first aid guidance in offline mobile environment, *arXiv preprint arXiv:2602.13229* (2026).

- [6] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense passage retrieval for open-domain question answering, in: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2020, pp. 6769–6781. doi:10.18653/v1/2020.emnlp-main.550.
URL <https://aclanthology.org/2020.emnlp-main.550/>
- [7] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, L. Qiu, LLMingua: Compressing prompts for accelerated inference of large language models, in: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 13358–13376. doi:10.18653/v1/2023.emnlp-main.825.
URL <https://aclanthology.org/2023.emnlp-main.825/>
- [8] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, D. Zhang, LLMingua-2: Data distillation for efficient and faithful task-agnostic prompt compression (2024). arXiv:2403.12968, doi:10.48550/arXiv.2403.12968.
URL <https://arxiv.org/abs/2403.12968>
- [9] F. Xu, W. Shi, E. Choi, RECOMP: Improving retrieval-augmented lms with compression and selective augmentation, arXiv preprint arXiv:2310.04408 (2023).
URL <https://arxiv.org/abs/2310.04408>
- [10] N. Chirkova, T. Formal, V. Nikoulina, S. Clinchant, Provence: Efficient and robust context pruning for retrieval-augmented generation, arXiv preprint arXiv:2501.16214 (2025).
URL <https://arxiv.org/abs/2501.16214>
- [11] R. Nogueira, K. Cho, Passage re-ranking with BERT, arXiv preprint arXiv:1901.04085 (2019).
URL <https://arxiv.org/abs/1901.04085>
- [12] BAAI, BGE-Reranker-v2: Bge reranker documentation, https://bge-model.com/bge/bge_reranker_v2.html, accessed 2026-07-09 (2024).
- [13] B. Jiang, One model is enough: Native retrieval embeddings from LLM agent hidden states, arXiv preprint arXiv:2603.08429 (2026).

doi:10.48550/arXiv.2603.08429.

URL <https://arxiv.org/abs/2603.08429>

- [14] Z. Li, T. Zhou, Your mixture-of-experts LLM is secretly an embedding model for free, in: The Thirteenth International Conference on Learning Representations, 2025.
URL <https://openreview.net/forum?id=eFGQ97z5Cd>
- [15] W. H. Raza, Y. Wen, A. B. Shah, Y. Shen, J. Martinez-Lemus, M. C. Schiess, T. M. Ellmore, R. Hu, X. Fu, Neuromoe++: Patient-adaptive multi-level multimodal fusion with mixture-of-experts for neurological disorder classification, *IEEE Transactions on Biomedical Engineering* (2026) 1–11doi:10.1109/TBME.2026.3691657.
- [16] W. H. Raza, A. B. Shah, Y. Wen, Y. Shen, J. D. M. Lemus, M. C. Schiess, T. M. Ellmore, R. Hu, X. Fu, Neuromoe: A transformer-based mixture-of-experts framework for multi-modal neurological disorder classification (2025). arXiv:2506.14970.
URL <https://arxiv.org/abs/2506.14970>
- [17] A. B. Shah, Y. Wen, R. Hu, J. Chen, J. L. Contreras-Vidal, X. Wu, X. Fu, Cogmoe: Signal-quality-guided multimodal moe for cognitive load prediction (2026).
- [18] W. H. Raza, M. Schiess, J. M. Lemus, T. M. Ellmore, C. Green, C. Soto, X. Fu, R. Hu, R2moe: Representation and expert selection dual-regularized mixture-of-experts for multimodal clinical data, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Findings*, 2026, pp. 5776–5785.
- [19] N. Muennighoff, L. Soldaini, D. Groeneveld, K. Lo, J. Morrison, S. Min, W. Shi, P. Walsh, O. Tafjord, N. Lambert, et al., OLMoE: Open mixture-of-experts language models, arXiv preprint arXiv:2409.02060 (2024). doi:10.48550/arXiv.2409.02060.
URL <https://arxiv.org/abs/2409.02060>
- [20] L. Bandarkar, C. Yang, M. Fayyaz, J. Hu, N. Peng, Multilingual routing in mixture-of-experts, in: The Fourteenth International Conference on Learning Representations, 2026.
URL <https://openreview.net/forum?id=ZoZR0x7tTD>

- [21] W. Fedus, B. Zoph, N. Shazeer, Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, *Journal of Machine Learning Research* 23 (120) (2022) 1–39.
URL <https://jmlr.org/papers/v23/21-0998.html>
- [22] Y. Li, B. Dong, F. Guerin, C. Lin, Compressing context to enhance inference efficiency of large language models, in: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, Singapore, 2023, pp. 6342–6353. doi:10.18653/v1/2023.emnlp-main.391.
URL <https://aclanthology.org/2023.emnlp-main.391/>
- [23] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, L. Qiu, LLMingua: Compressing prompts for accelerated inference of large language models, in: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, Singapore, 2023, pp. 13358–13376. doi:10.18653/v1/2023.emnlp-main.825.
URL <https://aclanthology.org/2023.emnlp-main.825/>
- [24] W. Wang, Y. Wang, Y. Fan, H. Liao, J. Guo, Quito: Accelerating long-context reasoning through query-guided context compression (2024). arXiv:2408.00274, doi:10.48550/arXiv.2408.00274.
URL <https://arxiv.org/abs/2408.00274>
- [25] Z. Wang, J. Araki, Z. Jiang, M. R. Parvez, G. Neubig, Learning to filter context for retrieval-augmented generation (2023). arXiv:2311.08377.
URL <https://arxiv.org/abs/2311.08377>
- [26] Sentenceembeddingoptimizer — llamaindex documentation, accessed: 2026-02-26 (2026).
- [27] Rankers — haystack documentation, <https://docs.haystack.deepset.ai/docs/rankers>, accessed: 2026-02-26 (2026).
- [28] C. Yoon, T. Lee, H. Hwang, M. Jeong, J. Kang, Compact: Compressing retrieved documents actively for question answering, in: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, Miami, Florida, USA, 2024, pp. 21424–21439. doi:10.18653/v1/2024.emnlp-main.1194.
URL <https://aclanthology.org/2024.emnlp-main.1194/>

- [29] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, C. D. Manning, Raptor: Recursive abstractive processing for tree-organized retrieval (2024). arXiv:2401.18059, doi:10.48550/arXiv.2401.18059.
URL <https://arxiv.org/abs/2401.18059>
- [30] LLMChainExtractor — langchain documentation (v0.3), accessed: 2026-02-26 (2026).
- [31] S. Robertson, H. Zaragoza, The probabilistic relevance framework: Bm25 and beyond, *Foundations and Trends in Information Retrieval* 3 (4) (2009) 333–389. doi:10.1561/15000000019.
URL <https://doi.org/10.1561/15000000019>
- [32] T. Joachims, Optimizing search engines using clickthrough data, in: *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2002, pp. 133–142. doi:10.1145/775047.775067.
URL <https://doi.org/10.1145/775047.775067>
- [33] C. J. C. Burges, From ranknet to lambdarank to lambdamart: An overview, Tech. Rep. MSR-TR-2010-82, Microsoft Research (2010).
- [34] B. Mitra, F. Diaz, N. Craswell, Learning to match using local and distributed representations of text for web search, in: *Proceedings of the 26th International Conference on World Wide Web, WWW '17, International World Wide Web Conferences Steering Committee*, 2017, pp. 1291–1299. doi:10.1145/3038912.3052579.
URL <https://doi.org/10.1145/3038912.3052579>
- [35] J. Guo, Y. Fan, Q. Ai, W. B. Croft, A deep relevance matching model for ad-hoc retrieval, in: *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management, CIKM '16*, Association for Computing Machinery, 2016, pp. 55–64. doi:10.1145/2983323.2983769.
URL <https://doi.org/10.1145/2983323.2983769>
- [36] C. Xiong, Z. Dai, J. Callan, Z. Liu, R. Power, End-to-end neural ad-hoc ranking with kernel pooling, in: *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '17*, Association for Computing Machinery, 2017, pp.

55–64. doi:10.1145/3077136.3080809.
URL <https://doi.org/10.1145/3077136.3080809>

- [37] N. Reimers, I. Gurevych, Sentence-bert: Sentence embeddings using siamese bert-networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, Association for Computational Linguistics, 2019, pp. 3982–3992. doi:10.18653/v1/D19-1410.
URL <https://aclanthology.org/D19-1410/>
- [38] Sentence Transformers, all-MiniLM-L6-v2, <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>, accessed: 2026-07-11 (2021).
- [39] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, H. Jégou, The faiss library, arXiv preprint arXiv:2401.08281 (2024). doi:10.48550/arXiv.2401.08281.
URL <https://arxiv.org/abs/2401.08281>